

Understanding P2P File Sharing (Torrent)

A Complete Beginner-Friendly Tutorial

When users download a file from websites like a software server, the file usually comes from **one central server**.

This is called:

Client-Server File Sharing

Example:

Your Computer → Server → File Download

But torrenting works differently.

Instead of downloading from one server, the file is downloaded from **many users simultaneously**.

This is called:

Peer-to-Peer (P2P) File Sharing

Torrenting is one of the most popular implementations of P2P.

This tutorial explains:

- What torrenting is
- How P2P works
- Torrent protocols
- Ports used
- Sequence of communication
- Step-by-step file sharing process
- Important networking concepts involved

1. What is P2P File Sharing?

P2P stands for:

Peer-to-Peer

In this model, every participant can act as:

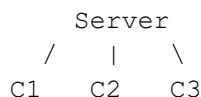
- Client
- Server

This means a computer can:

- Download file pieces
- Upload file pieces to others

simultaneously.

Traditional Client-Server Model

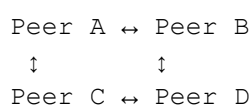


One server sends data to everyone.

Problems:

- Server overload
- High bandwidth cost
- Single point of failure

P2P Model



Everyone shares with everyone.

Advantages:

- Faster downloads
- No central bottleneck
- Better scalability

2. Important Torrent Terminology

Torrent File (.torrent)

A small metadata file containing information about:

- File name
- File size
- Piece hashes
- Tracker URLs

It does NOT contain the actual file.

Example:

`ubuntu.iso.torrent`

Magnet Link

A link containing torrent metadata.

Example:

`magnet:?xt=urn:btih:...`

It allows downloading without needing a .torrent file.

Peer

A computer participating in file sharing.

Seeder

A peer that has the complete file.

Uploads only.

Leecher

A peer still downloading.

Usually uploads downloaded pieces too.

Swarm

The group of all peers sharing a specific torrent.

Piece

Files are split into small chunks.

Example:

A 1 GB file may be split into:

`1024 pieces of 1 MB each`

Tracker

A server that helps peers discover each other.

It does NOT host the file.

3. Main Protocol Used

Torrenting primarily uses:

BitTorrent Protocol

This protocol defines how peers:

- Discover each other
- Exchange pieces
- Verify integrity
- Upload/download data

4. Ports Used in Torrenting

Common ports:

Purpose	Port
Tracker communication (HTTP)	80
Tracker communication (HTTPS)	443
Peer communication	6881–6889
DHT communication	Variable UDP ports

Modern torrent clients often use:

49152–65535

(random high ports)

Protocol Types

Protocol	Usage
TCP	Peer data transfer
UDP	DHT, tracker communication
HTTP/HTTPS	Tracker requests

5. Step-by-Step Torrent Download Process

Let us assume a user wants to download:

`ubuntu.iso`

using torrent.

Step 1: User Opens Torrent File or Magnet Link

The user clicks:

`ubuntu.torrent`

or

`magnet link`

The torrent client opens.

Examples of torrent clients:

- qBittorrent
- Transmission
- Deluge

What Client Reads

The client extracts:

- Info hash
- Tracker addresses
- File details
- Piece hashes

Step 2: Client Contacts Tracker

The client sends a request to tracker.

Example:

`GET /announce`

This includes:

- Client ID

- Port number
- Info hash
- Download progress

Tracker replies with peer list:

Peer 1: 192.168.x.x

Peer 2: 54.x.x.x

Peer 3: 103.x.x.x

Purpose of Tracker

Tracker helps peers discover one another.

It does not send file data.

Think of tracker as:

Contact Directory

Step 3: DHT Lookup (Optional)

Modern clients also use:

DHT (Distributed Hash Table)

This enables peer discovery without tracker.

It is decentralized.

Client asks DHT network:

Who has this file?

Other nodes reply with peer addresses.

Why DHT?

If tracker is offline, swarm still works.

Step 4: Peer Connection Initiation

Client selects peers.

It opens TCP connections.

Example

Your client connects to:

Peer A → Port 51413

Peer B → Port 60234

Peer C → Port 6881

TCP Three-Way Handshake Happens

Same as web browsing:

SYN

SYN-ACK

ACK

Connection established.

Step 5: BitTorrent Handshake

After TCP connection, peers perform BitTorrent handshake.

This verifies:

- Both support BitTorrent

- Both share same torrent

Handshake contains:

- Protocol identifier
- Reserved bytes
- Info hash
- Peer ID

Example:

```
BitTorrent protocol
Info Hash: abc123...
Peer ID: -qB1234-
```

Why This Matters

Prevents downloading wrong files.

Ensures peers are in same swarm.

Step 6: Exchange Bitfields

Peers tell each other which pieces they have.

Example:

Peer A:

```
11110000
```

Meaning:

Has pieces:

1, 2, 3, 4

Missing:

5, 6, 7, 8

Your client compares available pieces across swarm.

Step 7: Piece Selection

Client chooses which piece to request.

Most clients use:

Rarest First Strategy

Download rarest available pieces first.

Why?

Prevents piece disappearance.

Improves swarm health.

Step 8: Piece Request

Client requests:

```
Request piece #57
```

Peer sends data block.

Step 9: Piece Verification

Each piece has SHA hash.

After receiving:

Client computes hash.

Compares with torrent metadata.

If match:

Piece accepted

If mismatch:

Piece rejected

Why Verification?

Prevents:

- Corruption
- Malicious modification
- Transmission errors

Step 10: Upload Begins Immediately

As soon as your client has valid pieces, it starts sharing them.

This is why torrent users upload while downloading.

You become:

Leecher + Partial Seeder

Step 11: File Completion

When all pieces are received:

Client reconstructs file.

Example:

Piece 1

Piece 2

Piece 3

...

Piece N

combined into:

ubuntu.iso

Step 12: Become Seeder

Once complete:

You continue uploading.

You are now:

Seeder

Complete Sequence Diagram

User opens magnet link

↓

Torrent client starts

↓

Read metadata

↓

Contact tracker / DHT

↓

Receive peer list

↓

Connect to peers

↓

TCP handshake

↓

BitTorrent handshake

↓
Exchange piece availability
↓
Request file pieces
↓
Verify piece hashes
↓
Assemble file
↓
Become seeder

6. How Torrent Achieves High Speed

Suppose file has 100 pieces.

Instead of:

One server → sends all 100

Torrent does:

Peer A → Pieces 1-20

Peer B → Pieces 21-40

Peer C → Pieces 41-60

Peer D → Pieces 61-80

Peer E → Pieces 81-100

Downloaded simultaneously.

This increases speed.

7. Choking and Unchoking

Torrent clients control upload bandwidth.

Choking

Temporarily refusing upload.

Unchoking

Allowing upload.

Clients prefer peers who:

- Upload back
- Respond quickly

This creates fairness.

Called:

Tit-for-Tat Mechanism

8. NAT Traversal Problem

Many users are behind routers.

Private IP example:

192.168.1.10

Not directly reachable.

Torrent clients solve this using:

- UPnP
- NAT-PMP
- Port forwarding
- Hole punching

Why Open Port Matters

If incoming port is blocked:

You can download

But may upload less efficiently

9. Encryption in Torrenting

Some clients support:

Protocol Encryption (PE)

Purpose:

- Obfuscate traffic
- Avoid ISP throttling

It is NOT strong privacy encryption.

For privacy, users often use VPN.

10. Torrent vs Traditional Download

Feature	Traditional Download	Torrent
Source	One server	Many peers
Speed	Limited by server	Distributed
Failure	Server crash stops download	Continues if peers exist
Cost	Server pays bandwidth	Shared among users

11. Real Network Packet Sequence

In Wireshark, torrent traffic often appears as:

```
DNS Query
DNS Response
Tracker HTTP Request
Tracker Response
TCP SYN
TCP SYN-ACK
TCP ACK
BitTorrent Handshake
Piece Requests
Piece Responses
Hash Verification
Upload/Download Traffic
```

12. Example Real Download Walkthrough

Suppose downloading Ubuntu Linux.

Step 1

User opens magnet link

Step 2

Client extracts:

```
Info Hash
```

Step 3

Contacts tracker

Step 4

Tracker returns:

50 peers

Step 5

Client connects to 15 fastest peers

Step 6

Requests different pieces from each

Step 7

Verifies pieces

Step 8

Assembles ISO

Step 9

Seeds to others

13. Advantages of Torrenting

- Efficient bandwidth usage
- High scalability
- Faster large file distribution
- No central bottleneck
- Fault tolerant

14. Disadvantages

- Requires active seeders
- NAT/firewall issues
- Can expose IP address
- Potential legal misuse
- Variable speeds

15. Easy Analogy for Students

Imagine 100 students need same book.

Traditional method:

One teacher photocopies 100 copies.

Slow.

Torrent method:

Teacher gives 10 pages to each student.

Students exchange pages among themselves.

Everyone gets full book faster.

That is torrenting.

Key Concepts Students Must Remember

BitTorrent uses:

- TCP
- UDP
- HTTP/HTTPS

Main Components

- Torrent file
- Tracker
- Peer
- Seeder
- Leecher
- Swarm
- Piece

Important Process

Discover peers
Connect
Handshake
Exchange pieces
Verify
Assemble
Seed

Final Summary

P2P torrenting works by dividing files into small pieces and downloading them from multiple peers simultaneously.

The process is:

1. Open torrent file/magnet link
2. Contact tracker/DHT
3. Discover peers
4. Establish TCP connections
5. Perform BitTorrent handshake
6. Exchange available pieces
7. Download pieces
8. Verify hashes
9. Reconstruct file
10. Upload to others

This decentralized approach makes file distribution efficient, scalable, and fault tolerant.